A Mini Project Report Of

PCC-201-COM : Data Structures

Submitted by

Yash Goraksh Mapari (SEC B - 20)

Under the Guidance of

Prof. Tejal Harshal Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the “Mini Project Report” submitted by Yash Goraksh Mapari (20), on “Implementation of Snake and Ladder using BFS” is work done by him and submitted during the 2026–2027 academic year, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Engineering, at Bharati Vidyapeeth’s College of Engineering, Lavale, Pune.

Prof. Tejal Harshal Patil
(Guide)

Dr. L. V. Patil
Head of Computer Engg. Department

ACKNOWLEDGEMENT

It gives us immense pleasure to express our gratitude and thanks to all those who have helped in the timely completion of the Mini-Project work.

We have great pleasure in expressing our deep sense of gratitude and indebtedness to our Guide, **Prof. Tejal Harshal Patil**, for her invaluable guidance, constant supervision and sustained encouragement throughout the tenure of this Mini Project work.

We would like to express our thanks to **Dr. L. V. Patil**, Head of Department, Department of Computer Engineering, Bharati Vidyapeeth's College of Engineering, Lavale, Pune, for his constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil**, Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune, for permitting us to take up the project work. We are also thankful to our parents and friends for their kind help and support throughout the course.

Yash Goraksh Mapari

Student Name



ABSTRACT

The Snake and Ladder game is a classic board game played on a numbered grid, usually of 100 squares, where a player advances by the roll of a dice, climbs up when landing on the foot of a ladder and slides down when landing on the head of a snake. A natural question is: *what is the minimum number of dice throws needed to reach the last square?*

In this project, the board is modelled as an unweighted, directed graph in which every square is a vertex and every possible dice outcome (1 to 6) is an edge. Because every move costs exactly one dice throw, the Breadth-First Search (BFS) algorithm — which explores a graph level by level — is guaranteed to find the minimum number of throws. A queue (collections.deque) stores the squares waiting to be explored and a visited array prevents revisiting squares.

The solution runs in O(N) time and O(N) space for a board of N squares and is implemented in Python using an object-oriented design with a menu-driven interface. The project strengthens understanding of graph modelling, BFS traversal, queue usage and shortest-path reasoning in data structures.

CONTENT

Sr. No.	Content	Page No.
	Acknowledgement	i
	Abstract	ii
	Content	iii
	List of Figures	iv
1	Introduction a) Overview b) Problem Statement c) Objective	1
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	4
5	System Design a) ER Diagram b) System Code	5 5 6
6	Result	8
7	Application	10
8	Future Scope	11
9	Conclusion	12
10	References	13

LIST OF FIGURES

Figure	Description	Page No.
Fig 3.1	Use Case Diagram	03
Fig 5.1	ER / Class Diagram	05
Fig 5.2	System Code	06
Fig 5.3	Sample Board Configuration	08
Fig 5.4	Program Output (Result)	08

INTRODUCTION

1.1 Overview

The project "Implementation of Snake and Ladder using BFS" demonstrates how a classic board game can be solved efficiently by applying graph-traversal techniques. The game is played on a board of 100 squares numbered from 1 to 100. A player starts at square 1 and moves forward by the number shown on a dice (1 to 6). If the player lands on the foot of a **ladder** the player climbs to its top, and if the player lands on the head of a **snake** the player slides down to its tail. The goal is to reach square 100.

This project does not simulate random play; instead it finds the **minimum number of dice throws** needed to win, assuming the player could choose the dice value on every throw. The board is converted into an unweighted graph: each square is a vertex, and from every square there are up to six outgoing edges, one for each dice outcome. If an edge lands on a snake head or a ladder foot, the edge is redirected to the snake's tail or the ladder's top. Since every edge has the same cost (one throw), Breadth-First Search yields the optimal answer.

The program is developed in Python using an object-oriented approach. A `deque` acts as the BFS queue, a visited array avoids repeated work, and a parent array allows the exact sequence of throws to be reconstructed. A menu-driven interface lets the user view the board, compute the minimum throws, display the shortest path and add new snakes or ladders interactively.

1.2 Problem Statement

To model a Snake and Ladder board of N squares as a graph and use the Breadth-First Search algorithm to determine the minimum number of dice throws, and the corresponding path, required to move from square 1 to square N, through a menu-driven Python program.

1.3 Objective

The main objective of this project is to design and implement a solver for the Snake and Ladder game in Python using BFS, demonstrating how a real-world puzzle can be mapped onto a graph and solved with a standard traversal algorithm.

Through this implementation, the project aims to strengthen the understanding of graph representation, the role of a queue in BFS, and why BFS gives shortest paths in unweighted graphs. By building a menu-driven, object-oriented program, the project also improves practical coding skills, logical thinking and problem-solving ability.

LITERATURE SURVEY

Finding the minimum number of moves in the Snake and Ladder game is a well-known problem in data structures and algorithms. It is a standard example of a *shortest path problem in an unweighted graph* and appears in textbooks, competitive-programming platforms and technical interviews (for example, LeetCode Problem 909: Snakes and Ladders). Several approaches have been discussed:

1. **Recursion / Depth-First Search (DFS):** Explores every possible sequence of dice rolls. Without pruning it has exponential time complexity and, because snakes can create cycles, it needs extra visited tracking. DFS also does not guarantee that the first path found to the end is the shortest.
2. **Dynamic Programming:** Stores the minimum throws for every square. It is difficult to apply directly because snakes move the player backwards, which breaks the simple left-to-right ordering DP relies on.
3. **Breadth-First Search (BFS):** Treats squares as vertices and dice outcomes as edges. BFS visits all squares reachable in 1 throw, then in 2 throws, and so on, so the first time the last square is reached the number of throws is minimal.

This project adopts the **BFS approach**. Each square is visited at most once and has at most six outgoing edges, so the time complexity is O(6N) = O(N) and the space complexity is O(N) for the visited array, parent array and queue.

The table below summarises the comparison of the approaches.

Approach	**Time Complexity**	**Gives Shortest Path?**
Plain recursion / DFS	Exponential	No (needs full search)
Dynamic Programming	O(N) (hard with snakes)	Only with careful ordering
Breadth-First Search	O(N)	Yes — guaranteed

Implementing this solver not only reinforces the relationship between graphs and queues, but also gives a hands-on framework for analysing time and space complexity, and highlights how foundational data structures solve practical problems.

UML DIAGRAM

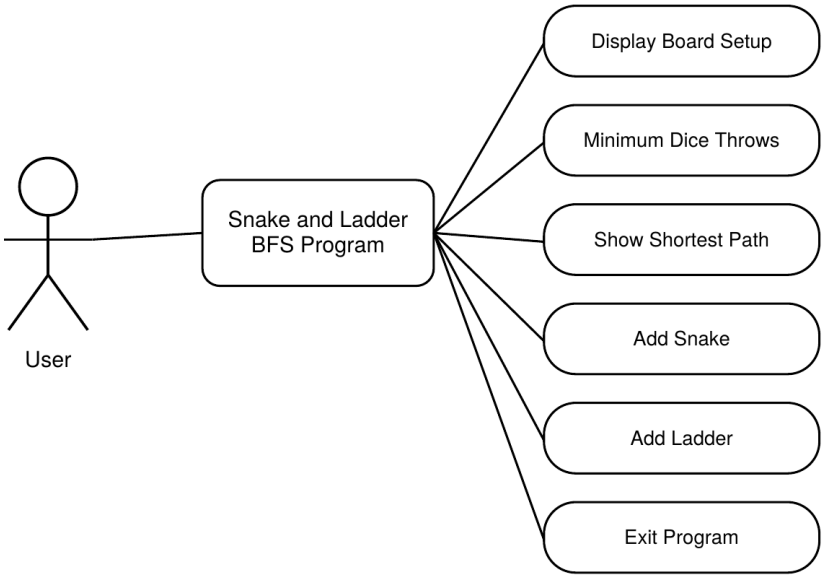


Fig 3.1: Use Case Diagram

SYSTEM REQUIREMENT

Hardware Requirements

Component	Minimum Requirement	Recommended
Operating System	Windows 7 / 8 / 10 / 11 (64-bit)	Windows 11 (64-bit)
Processor	Intel Core 2 Duo or equivalent	Intel Core i3 or higher
RAM	512 MB	2 GB or higher
Processor Speed	2.3 GHz or higher	3.0 GHz or higher
Hard Disk	100 MB free space	1 GB or more

Software Requirements

Software	Version / Notes
Python	3.7 or higher
IDE / Environment	Jupyter Notebook / PyCharm / VS Code
Required Libraries	None (uses built-in collections.deque only)

SYSTEM DESIGN

Entity Relationship Diagram

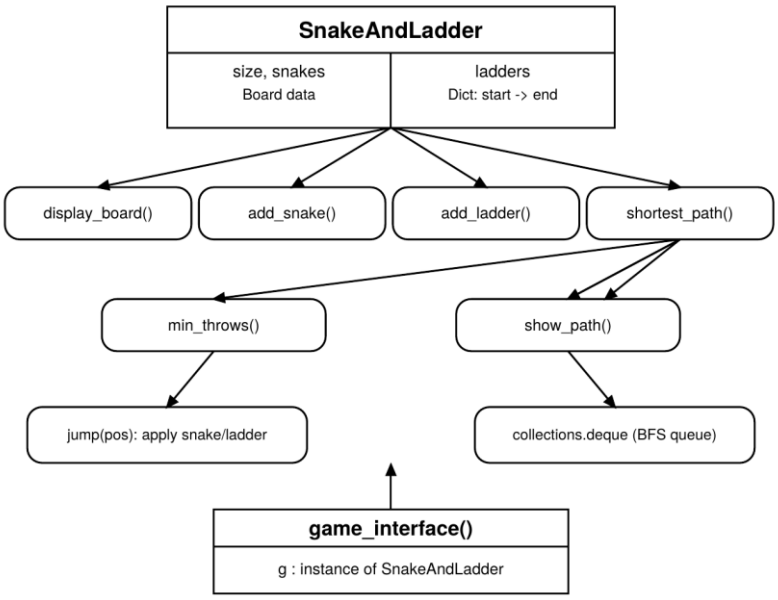
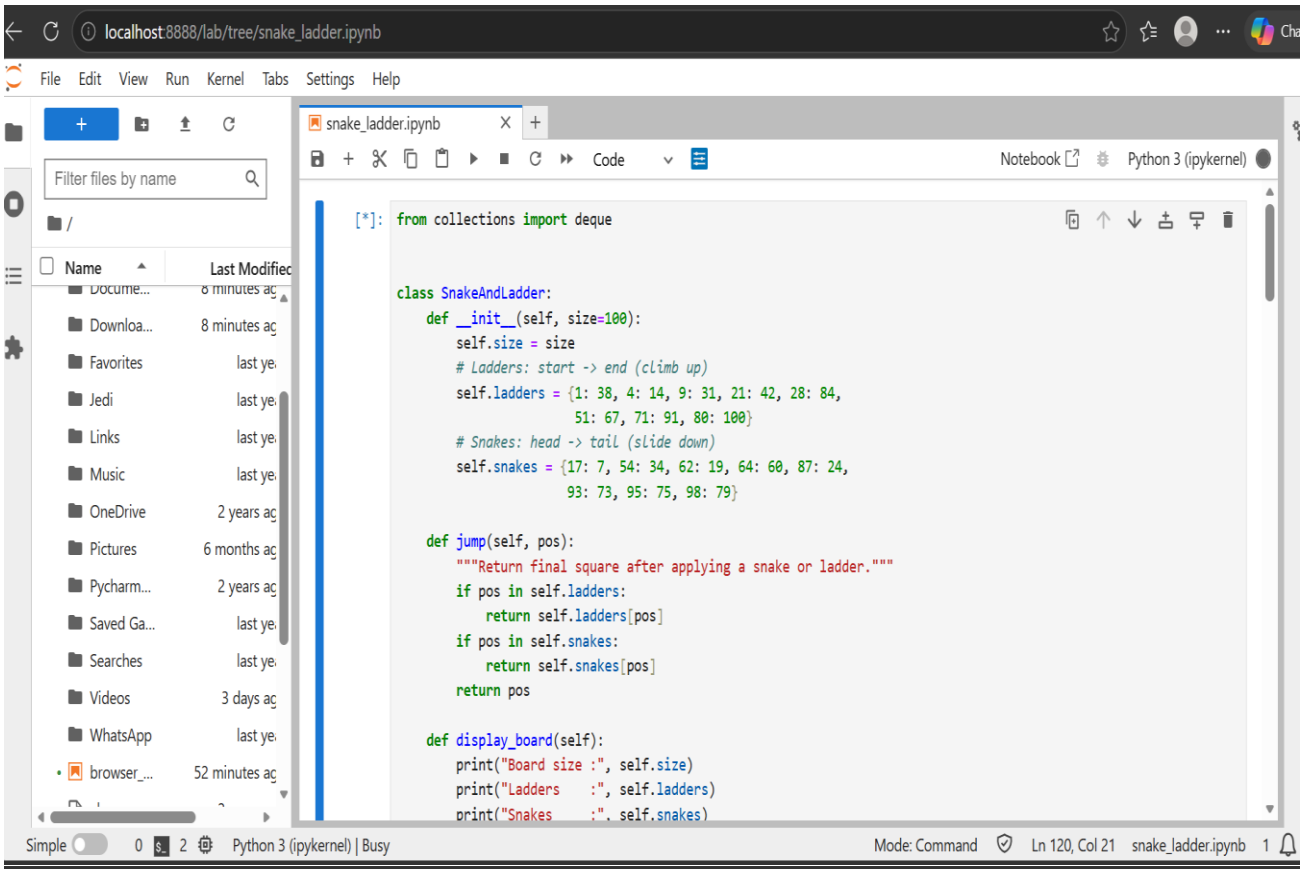


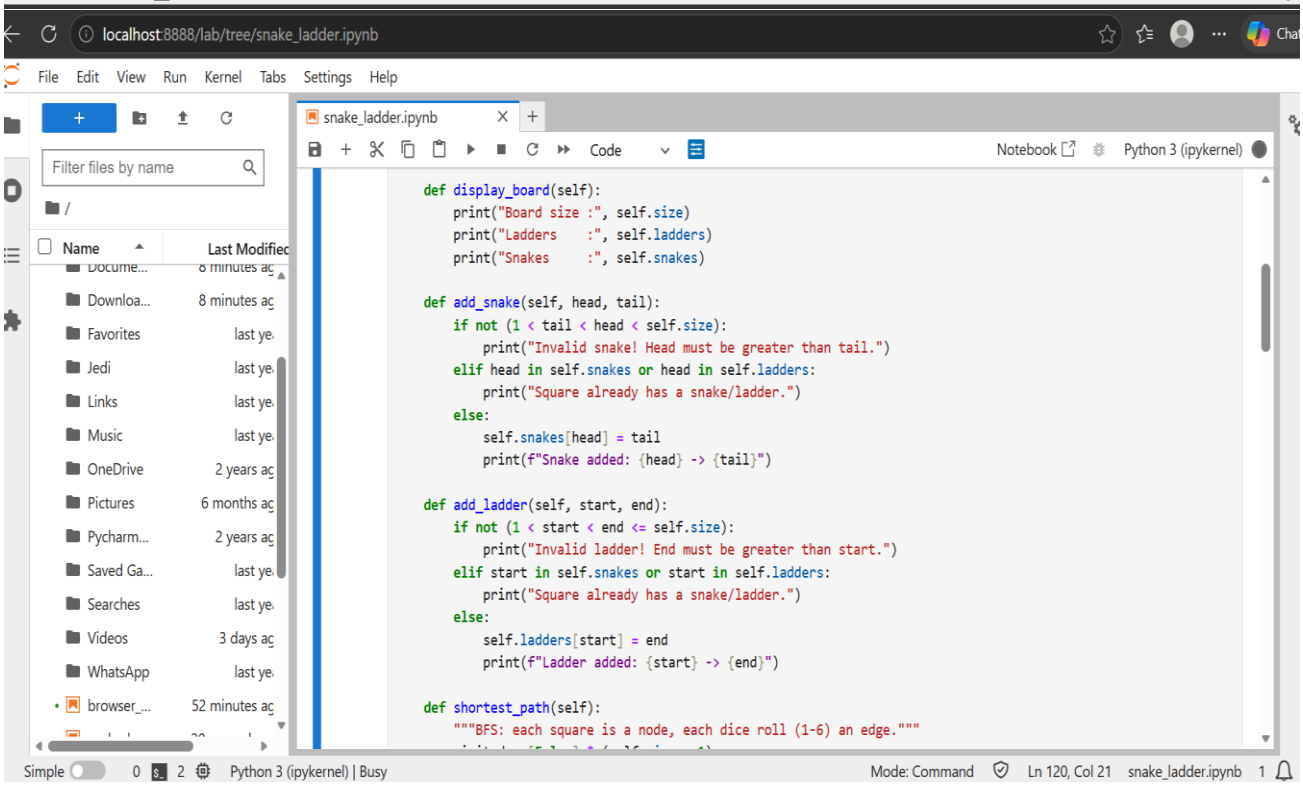
Fig 5.1: ER Diagram

Algorithm (BFS)

1. Create arrays *visited[1..N]* and *parent[1..N]*; push square 1 with 0 throws into the queue and mark it visited.
2. Remove the front square *pos* from the queue. If *pos* = *N*, stop — the throws count is the answer.
3. For each dice value 1 to 6, compute *next* = *pos* + *dice*; skip it if it exceeds N.
4. If *next* has a snake head or ladder foot, replace it with the tail or top (*jump*).
5. If *next* is not visited, mark it visited, store its parent and add it to the queue with throws + 1.
6. Repeat until the queue is empty; if N was never reached, report that it is unreachable.

SYSTEM CODE


localhost:8888/lab/tree/snake_ladder.ipynb
File Edit View Run Kernel Tabs Settings Help
snake_ladder.ipynb
Code
Notebook
Python 3 (ipykernel)
Filter files by name
Name
Last Modified
[*]:
from collections import deque
class SnakeAndLadder:
def __init__(self, size=100):
self.size = size
Ladders: start -> end (climb up)
self.ladders = {1: 38, 4: 14, 9: 31, 21: 42, 28: 84,
51: 67, 71: 91, 80: 100}
Snakes: head -> tail (slide down)
self.snakes = {17: 7, 54: 34, 62: 19, 64: 60, 87: 24,
93: 73, 95: 75, 98: 79}
def jump(self, pos):
"""Return final square after applying a snake or ladder."""
if pos in self.ladders:
return self.ladders[pos]
if pos in self.snakes:
return self.snakes[pos]
return pos
def display_board(self):
print("Board size :", self.size)
print("Ladders :", self.ladders)
print("Snakes :", self.snakes)
Simple
Python 3 (ipykernel) | Busy
Mode: Command
Ln 120, Col 21
snake_ladder.ipynb


localhost:8888/lab/tree/snake_ladder.ipynb
File Edit View Run Kernel Tabs Settings Help
snake_ladder.ipynb
Code
Notebook
Python 3 (ipykernel)
Filter files by name
Name
Last Modified
def display_board(self):
print("Board size :", self.size)
print("Ladders :", self.ladders)
print("Snakes :", self.snakes)
def add_snake(self, head, tail):
if not (1 < tail < head < self.size):
print("Invalid snake! Head must be greater than tail.")
elif head in self.snakes or head in self.ladders:
print("Square already has a snake/ladder.")
else:
self.snakes[head] = tail
print(f"Snake added: {head} -> {tail}")
def add_ladder(self, start, end):
if not (1 < start < end <= self.size):
print("Invalid ladder! End must be greater than start.")
elif start in self.snakes or start in self.ladders:
print("Square already has a snake/ladder.")
else:
self.ladders[start] = end
print(f"Ladder added: {start} -> {end}")
def shortest_path(self):
"""BFS: each square is a node, each dice roll (1-6) an edge."""
Simple
Python 3 (ipykernel) | Busy
Mode: Command
Ln 120, Col 21
snake_ladder.ipynb

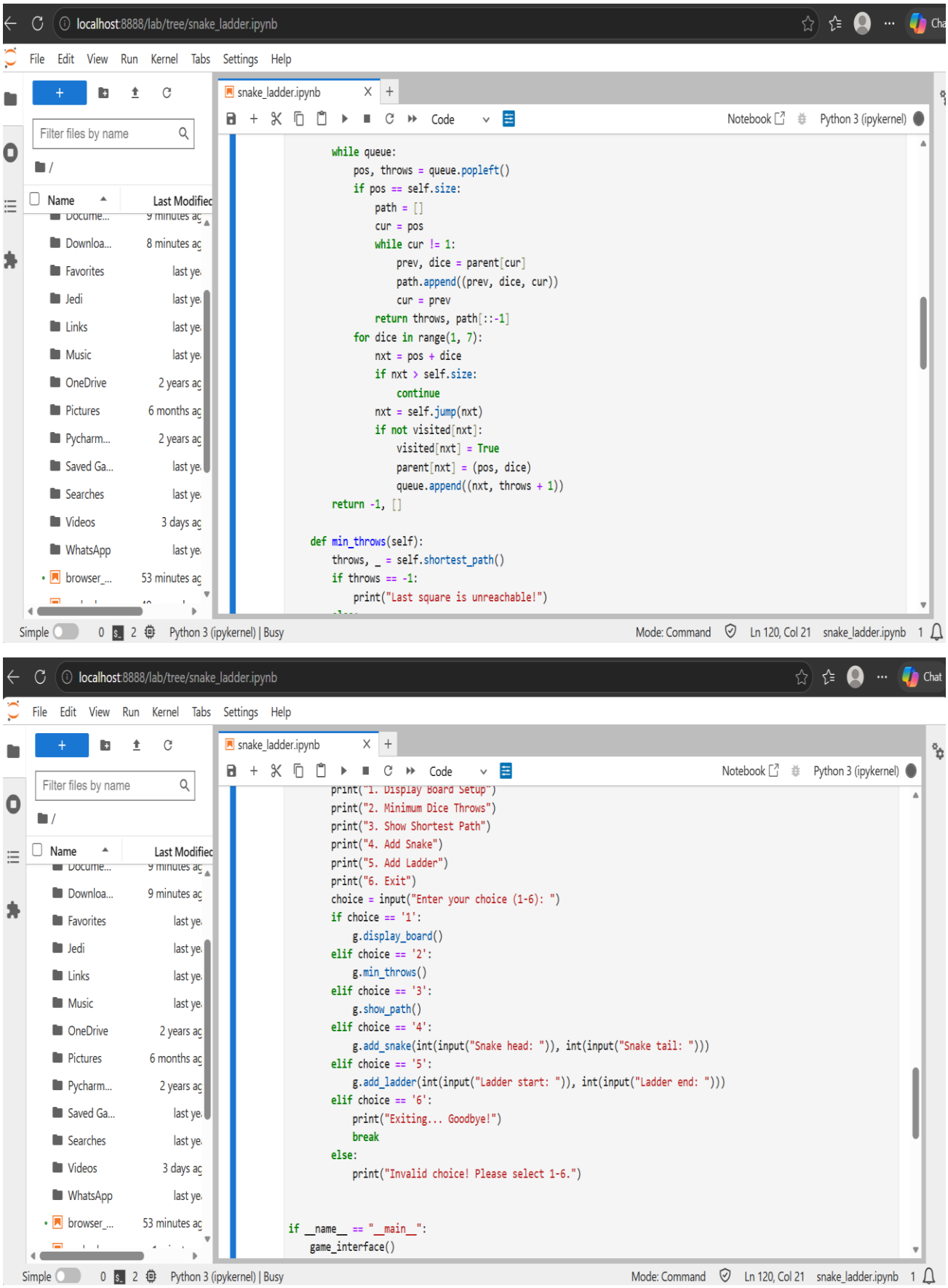
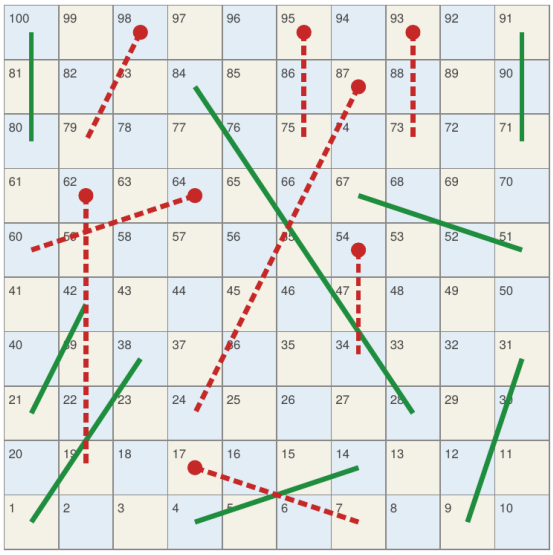


Fig 5.2: System Code

RESULT



Green = Ladder (climb up) Red = Snake (slide down)

Fig 5.3: Sample Board Configuration (default snakes and ladders)

```
=== Snake and Ladder (BFS) ===
1. Display Board Setup
2. Minimum Dice Throws
3. Show Shortest Path
4. Add Snake
5. Add Ladder
6. Exit
Enter your choice (1-6): 1
Board size : 100
Ladders    : {1: 38, 4: 14, 9: 31, 21: 42, 28: 84, 51: 67, 71: 91, 80: 100}
Snakes     : {17: 7, 54: 34, 62: 19, 64: 60, 87: 24, 93: 73, 95: 75, 98: 79}

=== Snake and Ladder (BFS) ===
1. Display Board Setup
2. Minimum Dice Throws
3. Show Shortest Path
4. Add Snake
5. Add Ladder
6. Exit
Enter your choice (1-6): 2
Minimum dice throws required: 7

=== Snake and Ladder (BFS) ===
1. Display Board Setup
2. Minimum Dice Throws
3. Show Shortest Path
4. Add Snake
5. Add Ladder
6. Exit
Enter your choice (1-6): 3
Shortest path ( 7 throws ):
  Throw 1: 1 --(dice 3)--> 14
  Throw 2: 14 --(dice 2)--> 16
  Throw 3: 16 --(dice 6)--> 22
  Throw 4: 22 --(dice 6)--> 84
  Throw 5: 84 --(dice 4)--> 88
  Throw 6: 88 --(dice 6)--> 94
  Throw 7: 94 --(dice 6)--> 100
```

Observation: On the default board the BFS solver reports a minimum of **7 dice throws** (1 → 14 → 16 → 22 → 84 → 88 → 94 → 100). After a new ladder 2 → 90 is added, the minimum drops to **3 throws** (1 → 90 → 94 → 100), confirming that the algorithm reacts correctly to changes in the board. Invalid menu choices are rejected with a message.

```
=== Snake and Ladder (BFS) ===
1. Display Board Setup
2. Minimum Dice Throws
3. Show Shortest Path
4. Add Snake
5. Add Ladder
6. Exit
Enter your choice (1-6): 4
Snake head: 99
Snake tail: 10
Snake added: 99 -> 10

=== Snake and Ladder (BFS) ===
1. Display Board Setup
2. Minimum Dice Throws
3. Show Shortest Path
4. Add Snake
5. Add Ladder
6. Exit
Enter your choice (1-6): 9
Invalid choice! Please select 1-6.

=== Snake and Ladder (BFS) ===
1. Display Board Setup
2. Minimum Dice Throws
3. Show Shortest Path
4. Add Snake
5. Add Ladder
6. Exit
Enter your choice (1-6): 5
Ladder start: 2
Ladder end: 90
Ladder added: 2 -> 90

=== Snake and Ladder (BFS) ===
1. Display Board Setup
2. Minimum Dice Throws
3. Show Shortest Path
4. Add Snake
5. Add Ladder
6. Exit
Enter your choice (1-6): 2
Minimum dice throws required: 3

=== Snake and Ladder (BFS) ===
1. Display Board Setup
2. Minimum Dice Throws
3. Show Shortest Path
4. Add Snake
5. Add Ladder
6. Exit
Enter your choice (1-6): 3
Shortest path ( 3 throws ):
  Throw 1: 1 --(dice 1)--> 90
  Throw 2: 90 --(dice 4)--> 94
  Throw 3: 94 --(dice 6)--> 100

=== Snake and Ladder (BFS) ===
1. Display Board Setup
2. Minimum Dice Throws
3. Show Shortest Path
4. Add Snake
5. Add Ladder
6. Exit
Enter your choice (1-6): 6
Exiting... Goodbye!
```

Fig 5.4: Program Output (Result)

APPLICATIONS

The BFS-based Snake and Ladder solver is mainly used in the following contexts:

1. Computer Science Education and Interviews

- **Concept Demonstration:** A classic example of converting a real-world problem into a graph and solving it with BFS.
- **Skill Assessment:** Frequently asked in technical interviews to test graph modelling, queue usage and shortest-path reasoning (for example, LeetCode 909).

2. General Shortest-Path and Game Applications

The same technique is used wherever the minimum number of steps in an unweighted state space is required:

Application	Description
Board Game Analysis	Finding the fastest possible win, checking fairness and balancing the placement of snakes and ladders when designing boards.
Game AI / Puzzle Solving	Solving sliding puzzles, mazes and word-ladder style problems where each move has equal cost.
Network Routing	Finding the route with the fewest hops between two nodes of a network.
Social Networks	Computing degrees of separation between two users (shortest friend chain).
Web Crawlers	Level-by-level exploration of web pages starting from a seed URL.

FUTURE WORK

1. Robustness and Performance Analysis

These improvements focus on making the code more reliable and understanding its efficiency.

- **Input Validation:** Reject non-numeric or out-of-range squares when adding snakes and ladders, and prevent a snake or ladder from forming a loop or blocking the last square.
- **Formal Complexity Analysis:** Document the O(N) time and space complexity, explaining that every square is enqueued at most once and has at most six outgoing edges.
- **Unit Tests:** Add tests using Python's unittest or pytest to cover edge cases such as:
  - A board with no snakes or ladders (answer = $\lceil (N-1)/6 \rceil$ throws).
  - A board where the last square is unreachable.
  - Snakes and ladders placed on squares 2–7 or in the last six squares.

2. Feature and Interface Expansion

These enhancements add useful functionality and improve the user experience.

- **Variable Board Size:** Allow boards other than 10 × 10 and dice with different numbers of faces.
- **Graphical Interface:** Draw the board and animate the shortest path using Tkinter, Pygame or a web front end.
- **Game Simulation:** Add a multi-player mode with random dice and compare actual play against the BFS optimum.
- **Type Hinting:** Use Python type hints (e.g., `def add_snake(self, head: int, tail: int) -> None`) to improve readability and enable static analysis.
- **Modularize the Interface:** Separate the SnakeAndLadder class from the game_interface() function into different modules for cleaner, more reusable code.

CONCLUSION

The SnakeAndLadder class provides a successful and instructive implementation of the Breadth-First Search algorithm for finding the minimum number of dice throws needed to complete a Snake and Ladder board. By modelling squares as vertices and dice outcomes as edges, the game is reduced to a shortest-path problem on an unweighted graph.

This implementation is particularly valuable in computer science for:

- Demonstrating how a real-world problem is converted into a graph and solved with a standard traversal.
- Achieving O(N) time and O(N) space complexity, with a parent array that reconstructs the exact path of throws.

The current code is functional and includes a basic menu interface. Future improvements should focus on robustness (input validation, edge-case handling), feature completeness (variable board size, graphical display) and formal testing to ensure it can serve as a reliable teaching and analysis tool.

REFERENCES

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

[2] LeetCode Problem 909: Snakes and Ladders.

[3] GeeksforGeeks: Snake and Ladder Problem (minimum dice throws using BFS).

[4] Python Software Foundation. *collections — Container datatypes (deque)*, Python 3 Documentation.